

Microprocessor based System Design Laboratory (Gx)

Department of Computer Science and Technology, IEST Shibpur

Experiment No. 2: Familiarization with 8085 Instruction Set

Date: July 27, 2026

Objective: To be familiar with the Instruction Set of 8085 Microprocessor through writing simple programs and testing them at the 8085 SDK (System Development Kit).

Preamble: By this time you are familiar with the SDK you are using in the laboratory. **Please ensure** that, **through the Monitor program** of the SDK, using the **keypad** and **display section**, you can now **interactively** do the following.

- Using “**EXAM MEM**” key (command) - To examine the contents of main memory and modify its contents (the RAM part, say, 8800H onward) to load a 8085 machine language program,
- Using “**GO ... EXEC**” keys (command) – To execute a program which is already available in the main memory of the SDK, and test the effect of the program for validating its expected result.

In general developing a program in the SDK, **for the time being**, will involve the following steps.

- Writing down the program in 8085 assembly language (as shown below) in your notebook.
- Translating the assembly language program of the previous step into machine language (shown below) by hand in your notebook.
- Load the machine language program in the available RAM section of your SDK (using “**EXAM MEM**” key)
- Execute the program (using “**GO <X X X X> EXEC**” keys)
- If the effect of the program is not as expected go to step i.

For example, the following table contains a 8085 program (in **assembly** as well as **machine language**) with documentation (comments).

Label (Name)	Assembly Instruction	Memory Address (Hex)	Machine Language (Hex)	Comment
	MVI B, 22H	8800	06	Register B is initialized with 22H
		8801	22	
	MVI C, 82H	8802	0E	Register C is initialized with 82H
		8803	82	
	MOV A, B	8804	78	Contents of B (that is, 22H) is copied to A
	STA 8840H	8805	32	Contents of A is stored (copied) to Memory location 2040H
		8806	40	8085 is Little Endian, lower order byte of 8840H, that is, 40H comes first before 88H, the higher order byte.
		8807	88	
	RST 3	8808	DF	RST 3 saves the current contents of PC into stack and loads PC with $3 \times 8 = 24$, that is, 18H. [Read from decription from 8085 Instruction Set Reference Encyclopedia] Hence, after RST 3 instruction, 8085 will execute the instruction at location 18H which, in the SDK, falls inside the ROM (containing the Monitor program). Actually, in your kit, the monitor program at 18H saves the CURRENT contents of the Registers somewhere in the RAM, so that, subsequently when you press “EXAM REG” key, the monitor program can show the-then contents of the registers, taking them from those locations of the memory.

After execution of the above program (GO-8-8-0-0-EXEC) you may check the contents of the memory location @8840H (should contain 22H) as well as contents of registers A (should contain 22H), B (should contain 22H) and C (should contain 82H).

In today's laboratory class **EACH OF YOU** have to write and test 8085 programs as specified below **in the format shown in the previous table**.

Get the program, that you have written in your notebook, signed by your teacher.

If you want to check the contents of the Registers after execution of your program, then end your program with “RST 3” instruction. After executing your program press the “EXAM REG” key; the monitor program would then show what the registers contained immediately before execution of RST 3 instruction.

However, after execution of your program, if you want to check the contents of some memory locations only, then you may end your program with “HLT” (halt) instruction. After executing your program, reset the SDK (“RESET**” key), press the “EXAM MEM” key; and check the contents of the concerned memory location.**

Assignment Programs

[Consult the **2-page Instruction Set** and **16-page Instruction Set Reference Encyclopedia** for choosing appropriate Instructions while writing programs]

- Let there be a C program as given below.

```
int main() {
    int x;
    x = 0x59; /* store 59H, ie, 89 in x */
    x = x + 0x37; /* add 37H ie, 55 to x */
}
```

In assembly language we will use either a register or some memory location for a variable. Hence come the following two possibilities.

- Load **59H** in a register (say, **B**) and add **37H** to the register (**B**). [Register B works as x]
 - Save **59H** in a memory location (say, **8900H**) and add **37H** to it (**8900H**) [Memory location **8900H** works as x]
- Let there be a C program as given below.

```
int main() {
    int x, y, z;
    x = 0x3A; /* store 3AH, ie, 58 in x */
    y = 0x1F; /* store 1FH, ie, 31 in y */
    z = x + y;
}
```

- Load two 8-bit numbers in 2 registers (say, **B** and **C**) and store their sum in a 3rd register (say, **D**).
 - Addition of two 8-bit numbers available in 2 memory locations (say, **8900H** and **8901H**) and storing their sum in a 3rd memory location (say, **8902H**). [Hint – Before executing the program store 2 numbers at **8900H** and **8901H**, then execute your program and after that check the contents at **8902H**.]
- Swapping the contents of two variables:** Consider the following C program.

```
int main() {
    int x=0x3A, y=0x1F, t;
    t = x;
    x = y;
    y = t;
}
```

Write and test 8085 Assembly Language Programs for the following.

- a. Swapping the contents of 2 registers (say, **B** and **C**).
- b. Swapping the contents of 2 memory locations (say, **8900H** and **8911H**).
- c. Swapping the contents of a register (say, **B**) and a memory location (say, **8900H**).
4. Storing the “**difference**” between two variables in another variable. Consider the following C program.

```
int main() {  
    int x = 0x3A, y = 0x1F, z;  
    if ( x > y) {  
        z = x - y;  
    } else {  
        z = y - x;  
    }  
}
```

For implementing the above C program in 8085 Assembly Language, the following rewrite makes it more comprehensible.

```
int main() {  
    int x = 0x3A, y = 0x1F, z;  
    if ( x > y) {  
        z = x - y;  
        goto next;  
    }  
    z = y - x;  
    next:  
}
```

[Hint: “if-else” type of statements can be realized using “**compare**” and “**jump**” types of instructions of 8085. “**compare**” type of instructions (say, **CMP B**) changes flag(s) in the Flag Register (depending on **(A) > (B)** or **(A) < (B)** or **(A) = (B)**). There are conditional jump instructions (Say, **JZ next**, jumps to “**next**” if **Z** flag is 1) which can be used for this purpose.]

Variable **x**, **y** and **z** can be realized by registers (Say, **B**, **C**, and **D**, respectively) or memory locations (Say, **8900H**, **8901H**, and **8902H**, respectively). Write 8085 programs for the above as per the following constraints.

- a. **x** and **y** are registers (say, **B** and **C**) and **z** is a memory location (Say, **8902H**).
- b. **x** and **y** are memory locations (say, **8900H**, **8901H**) and **z** is a register (Say, **D**).
- c. **x** is a register (say, **B**), **y** is a memory location (say, **8901H**) and **z** is a register (Say, **D**).
- d. **x** is a register (say, **B**), **y** is a memory location (say, **8901H**) and **z** is a memory location (Say, **8902H**).
5. Adding contents of 10 consecutive memory locations (starting from address **8900H**) and storing the sum at address **890AH**. This is, as if, adding the sum of elements of an array of 10 integers.

[Hint: A “loop” (“while”, “for”, etc) can be achieved through “if-else” and “goto” (that is, “jump”) statements also.]

6. The starting address (say, **8A00H**) of an array of 8-bit integers is available at address **8900H** (that is, location@**8900H** contains **00H**, and location@**8901H** contains **8AH**). The number of elements of the array is available at **8902H**. Find the sum of the elements of the array and store the sum at address **8903H**.
7. Refer to program 6 above. Storing sum of elements in 1 Byte is likely to lead to overflow and wrong result. Modify the program so that the sum is computed as a 2-Byte number in a register-pair (say **DE**) and finally stored in the memory using 2 locations (say, @ **8903H** – **8904H**).